AI 创新大赛 · 结课汇报

# AI for Comm 项目结课汇报书

通信领域多智能体科研协作系统

编制日期：2026-09-03 · 项目周期：2026-07-28 至 2026-09-02（约五周） · 编制：周行远（架构师）

运行环境：阿里云专有云 OpenTrek Agent Dev（智能体工厂）工作空间「AI For Comm」+ 本地 Python 编排层

# 目录


**01 项目功能总览**

1.1 系统定位
1.2 功能清单
1.3 模式一：快问快答
1.4 模式二：六步科研流水线
1.5 报告产出的固定形态
1.6 交付形态

**02 系统架构与智能体之间的通信**

2.1 分层架构：大脑在平台，手脚在本地
2.2 拓扑选择：星型
2.3 通信通道之一：平台调用协议（纵向，本地 ↔ 平台）
2.4 通信通道之二：内存消息总线（横向，智能体 ↔ 智能体）
2.5 通信通道之三：工具回环协议（智能体 ↔ 本地工具）
2.6 可靠性设计：故障 → 检测 → 处置
2.7 配置与可观测

**03 三个核心机制**

3.1 机制一：仿真真执行闭环
3.2 机制二：三级校验与闭环修正
3.3 机制三：分段统稿与高审美渲染
3.4 六道质量门

**04 七个智能体逐个详述**

4.1 总览表
4.2 math_agent：信息论与数学推导
4.3 comm_agent：3GPP 协议与组网（兼默认规划者）
4.4 hw_agent：FPGA / Verilog 硬件实践
4.5 sim_agent：蒙特卡洛仿真与数值验证
4.6 lit_review_agent：公网 + 知识库双源文献调研
4.7 verify_agent：三级校验
4.8 report_agent：统稿主编
4.9 分工边界：谁向谁要什么

**05 平台资产盘点**

5.1 智能体资产（7 个，全部在线）
5.2 知识库与本地知识资产
5.3 本地工具层（7 个工具，最小权限白名单）
5.4 技能资产（3 个，已上传平台技能中心）
5.5 提示词资产（13 份手册）
5.6 评测与留痕资产

**06 测试与量化验证**

6.1 测试轮次总账
6.2 评测集设计（38 题）
6.3 摸底成绩：35/37
6.4 离线回归矩阵
6.5 综合多智能体跑批
6.6 平台侧核对留痕（29 张截图，docs/验证记录图片/）

**07 后续规划**

7.1 把物理层信号处理链路纳入研究与验证范围
7.2 公网部署形态

附录 A · 最终提交材料里有什么

A.1 一级结构
A.2 项目文档\
A.3 演示成果\
A.4 代码\

附录 B · 数字复核索引

# 01 项目功能总览

## 1.1 系统定位

系统接收一句通信领域的自然语言输入，自动完成路由判定、智能体调用与本地工具执行，输出两类结果之一：一段带依据的专业答案，或一份含真实仿真图表、经独立校验的研究报告。

系统提供两种工作模式，共用同一套智能体资产与本地工具层：

| 模式 | 输入 | 内部动作 | 输出 | 典型耗时 |
|---|---|---|---|---|
| ① **快问快答**（单智能体直答） | 一个具体问题 | 关键词打分路由 → 单个专家作答 → 按需回环调用本地工具 | 结构化答案 + 工具使用标识 | 35 ~ 250 秒 |
| ② **科研任务**（六步流水线） | 一个研究课题 | 拆解 → 文献综述 → 多专家并行论证 → 交叉校验 → 闭环修正 → 分段统稿 → 渲染 | HTML 研究报告（内嵌真实仿真图） | 5 ~ 14 分钟 |

两种模式共用同一个 AgentNode 对象、同一份 `config.json`、同一套 `tools/` 工具白名单：快问快答是流水线第 3 步的单点形态，流水线是快问快答的多点编排形态。

## 1.2 功能清单

| # | 功能 | 入口 | 触发方式 | 产出 |
|---|---|---|---|---|
| 1 | 单智能体直答 | Web `/` · CLI `python main.py` | 自然语言问题，自动路由 | 答案正文 + 回答者标识 + 工具徽标 |
| 2 | 带真实计算的直答 | 同上 | 问题命中工具可解范围 | 答案内嵌符号计算与仿真的真实输出 |
| 3 | 推导+校验双节点问答 | CLI（数学类自动进入） | 关键词命中数学类 | 推导全文 + 校验意见，不通过自动重试 ≤3 轮 |
| 4 | 文献综述单点问答 | CLI（含"综述/文献/调研"） | `LitReviewAgent.survey()` | 公网+平台双源综述 + 参考文献 |
| 5 | 仿真五步法单点问答 | CLI（含"仿真/BER/蒙特卡洛"） | 自动追加一次性交付指令 | 方案 / 代码 / 理论对照 / 运行说明 |
| 6 | 六步科研流水线 | Web `/research` · CLI `科研: <课题>` | 一句课题 | HTML 研究报告 + 若干仿真图 |
| 7 | 流水线实时进度可视化 | Web `/research` | SSE `/api/pipeline/progress/<id>` | 六步状态灯 + 细节文案 + 协作网络动画 |
| 8 | 报告库（在线查看/下载） | Web `/reports` | 扫描 `outputs/research_*.html` | 列表、新窗口查看、附件下载 |
| 9 | 智能体监控看板 | Web `/monitor` | 读取配置与报告统计 | 7 智能体在线状态、知识库、报告数量 |
| 10 | 平台连接与凭证管理 | Web `/settings` | 表单保存 → 校验 → 热写 `config.json` | 免改代码切换智能体版本与知识库 |
| 11 | 无人值守批量跑研究 | CLI / 容器 | `python main.py` 脚本化 | 报告落盘 `outputs/` |
| 12 | 能力评测（38 题集） | `tests/run_eval.py` | 按科或按模式跑批 | HTML + CSV 评测结果 |
| 13 | 人工评分表生成 | `tests/make_score_sheet.py` | 读评测结果 | 四维打分表 HTML |
| 14 | 知识库源文档打包 | `build_kb_json.py` | 命令行 | 平台可导入的知识库 JSON |
| 15 | 容器化部署 | `Dockerfile` + `docker-compose.yml` | `docker compose up -d` | 7860 端口常驻服务 |

### 1.3 模式一：快问快答

**路由规则。** 路由是一张关键词打分表（`web/routes/helpers.py`），问题文本按命中次数为每个智能体计分，取最高分；全部未命中时落到默认路由 `comm_agent`：

| 目标智能体 | 触发关键词（不区分大小写） | 对应职责 |
|---|---|---|
| `math_agent` | AWGN、SNR、BER、香农、信道容量、互信息、熵、编码、调制、公式、推导、计算 | 信息论与数学推导 |
| `comm_agent` | GTP、S1AP、NAS、3GPP、协议、信令、组网、RRC、切换、5G、4G、LTE | 3GPP 协议与信令 |
| `hw_agent` | Verilog、FPGA、Vivado、硬件、时序、综合、RTL、模块 | FPGA / Verilog 实现 |
| `sim_agent` | 仿真、蒙特卡洛、模拟、曲线、误码率仿真 | 数值仿真验证 |
| `lit_review_agent` | 综述、文献、调研 | 文献综述（CLI 侧优先判定） |

两条入口的路由实现有意不同：Web 走纯打分制，稳定且适合演示；CLI 额外做两件事——命中"综述/文献"时直接调用 `LitReviewAgent.survey()`；命中数学类或无匹配时调用 `orchestrator.dual_verify()`，即 math_agent 推导 → verify_agent 审稿 → 不通过则修正重提的双节点闭环（最多 3 轮）。

**单答同样带本地工具。** 每次提问都经过 `ask_with_tools()`：驱动器先按问题关键词本地预跑 1~2 个离线工具（标准误码率曲线、香农容量、本地知识库摘录），把真实数值连同工具说明书一并交给智能体；智能体可直接引用这些数值，也可按协议申请更多工具，本地执行后结果回喂，再定稿（协议见 2.5）。前端据此在答案下方显示工具徽标。

### 1.4 模式二：六步科研流水线

| 步 | 名称 | 执行者 | 输入 | 输出 | 失败时的降级动作 |
|---|---|---|---|---|---|
| 1 | 问题拆解 | 规划者角色（候选 `comm_agent` → `math_agent`） | 一句课题 | ≤5 个子问题的 JSON，标注归属专家 | 解析失败→整题交给 comm；候选空回复→换下一候选 |
| 2 | 文献调研 | `lit_review_agent` | 课题 + 子问题清单 | 公网（arXiv/Semantic Scholar）+ 平台知识库双源综述 | 公网不可用→退化为纯平台库；异常→标注降级并继续 |
| 3 | 并行咨询专家 | math / comm / hw / sim | 文献综述 + 各自子问题 | 各专家完整论证 + 自主申请的工具清单 | 单专家失败只警告不中断；sim 有三级提示词退让 |
| 3.5 | 仿真代码真执行 | 本地沙箱 | sim 答案中的 Python 代码 | stdout + PNG 图表（自动回收） | 未提取到代码→记录原因，不编造结果 |
| 4 | 交叉校验 | `verify_agent` | 综述节选 + 全部专家论证 | 【通过】/【有条件通过】/【不通过】三级意见 | 平台异常重试 1 次，仍失败→标注跳过并继续出报告 |
| 4.5 | 闭环修正 | 原专家 + `verify_agent` | 评级为【不通过】的条目 | 修正后的论证 + 二次评级（≤2 轮） | 轮次耗尽→保留现状并在报告中如实标注 |
| 5 | 分段统稿 | `report_agent` + 本地合并 | 每位专家的完整论证（每次只送一位） | 固定 7 字段 JSON | 分节失败→原文直录；全局失败→本地补齐；平台全挂→机械拼装 |
| 6 | 报告渲染 | 本地渲染引擎 | 统稿 JSON + 全部原始材料 + 消息轨迹 | `outputs/research_<时间戳>.html` | 渲染异常→记录错误，前序产物不丢 |

两条设计约束：

- **同一专家至多领一道子问题。**规划者若拆出两道同归属的子问题，第二道并入同一次提问作为补充，不并发开两个会话，以规避平台同会话并发锁。
- **并发度固定为 2。**同时打开的网关 SSE 连接数限制在 2，超出会触发网关限流。

### 1.5 报告产出的固定形态

流水线产出的每份报告为单文件 HTML，自带样式与内嵌图片，可离线打开、可整文件转发，章节结构固定：

| 章节 | 内容 | 数据来源 |
|---|---|---|
| 封面 | 课题、执行摘要、关键词、耗时与参与智能体 | report_agent `summary` / `keywords` |
| Ⅰ 协作链路 | 各环节产出材料的字符量 | 本地 `steps` 消息轨迹 |
| Ⅱ 研究背景与文献综述 | 综述全文，含来源标注与参考文献 | lit_review_agent |
| Ⅲ 核心发现 | 每位专家一节正文 + 该节仿真图表 + 可展开的专家原文 | report_agent 分节统稿 + 专家原文 |
| Ⅳ 交叉分析 | 各专家结论是否互相印证、理论值与仿真值是否一致 | report_agent `cross_analysis` |
| Ⅴ 校验意见 | 三级评级与逐项核对结论，原样转述 | verify_agent |
| Ⅵ 综合结论与展望 | 结论以评级开头 + 后续可执行验证建议 | report_agent `conclusion` / `outlook` |
| 附录 | 协作过程留痕（消息轨迹表 + 材料体量与全文位置） | 本地编排层 |

### 1.6 交付形态

- **Web 应用**：Flask，六个中文页面（`/` 实时对话、`/research` 发起研究、`/reports` 报告库、`/reports/view/<file>` 在线查看、`/monitor` 监控、`/settings` 设置）；页面模板在 `web/templates/`，样式在 `web/static/css/style.css`，交互脚本在 `web/static/js/`（六个页面各一份），设计系统为深海军蓝 + 琥珀金，配自研 logo。
- **命令行**：`python main.py`，双模式文本交互，适合无人值守与录屏。
- **容器**：`Dockerfile` + `docker-compose.yml`，内网演示环境常驻 7860 端口。
- **报告归档**：`outputs/` 目录累计研究报告 17 份，仿真图表在 `outputs/sim/`。

# 02 系统架构与智能体之间的通信

本章回答一个核心问题：七个运行在远端平台上的智能体彼此不能直接通信，系统如何将它们组织为协作整体。

## 2.1 分层架构：大脑在平台，手脚在本地




分工原则：需要稳定、可审计、真算真跑的能力放本地，需要模型能力、知识检索、版本治理的能力放平台。两侧之间只有一个接口——`core/agent_node.py` 中约 200 行的 HTTP/SSE 客户端。

## 2.2 拓扑选择：星型

点对点的智能体互调在本平台不可行，原因有三条硬约束：

1. 角色对话型智能体不能被另一个智能体调用，平台不提供 agent-as-tool 的互挂能力；
2. 每次调用都是无记忆的，一次 `/run` 结束即完成使命，无法在其内部等待第三方回复；
3. 平台没有回调或 webhook 入口，不会主动把消息推给本地。

因此系统采用星型拓扑：编排器（`ResearchPipeline` + `Orchestrator` + `MessageBus`）是全图唯一枢纽，所有智能体只与编排器相连，任意两个智能体之间的"对话"实际上是编排器把一方产出注入另一方的下一次提问。




代价是多一次中枢搬运，收益是协作过程完全可记录、可重放、可降级，报告附录的"协作过程留痕"直接由此生成。

### 2.3 通信通道之一：平台调用协议（纵向，本地 ↔ 平台）

所有智能体共用一套调用协议，由 `AgentNode` 封装。平台侧接口文档见 `docs/平台参考资料/`（创建会话、创建连接、发起 agent 调用、发起一次对话、知识库检索共 5 份）。

**第一步 建会话**

```
POST {base}/createSession
Body: { "agentCode": "<UUID>", "agentVersion": "<版本编码>" }
Resp: { "data": "<uniqueCode>" }      ← 即 sessionId，一个 AgentNode 一个专属会话
```

`base` 为 `http://10.128.203.200:80/sfm-agent-studio/sfm-api-gateway/gateway/agent/api`，鉴权统一为 `Authorization: Bearer <APP_KEY>`。

**第二步 发起对话（流式）**

```
POST {base}/{agentCode}/run
Body: { "agentVersion": "...", "sessionId": "...", "stream": true, "delta": true,
        "input": "<提示词>", "kbIds": ["..."] }      ← 有 kbIds 时平台自动做知识库检索
```

**第三步 解析 SSE 流。**平台以 `data:{...}` 逐帧推送，本地按对象类型分流：

| SSE 对象 | 含义 | 本地处理 |
|---|---|---|
| `thought.delta` | ReAct 思考过程增量 | 默认丢弃；`collect_thought=True` 时缓存备用 |
| `message.delta` | 旧版回答增量（累计） | 计入终答 |
| `text.delta` | 新版回答增量（累计） | 计入终答 |
| `complete` | 流结束 | 立即退出读取 |

终答取用采用双保险：若 `text.delta` / `message.delta` 拼接结果为空或过短，自动回退到缓存的 `thought.delta`，避免把过渡语当成答案。

**第四步 会话无记忆 → 上下文由本地重建。**平台不保留上一轮，因此工具回环、闭环修正等多轮交互，本地都必须把"原始问题 + 上一轮回答 + 工具结果"整段重新拼进新的 `input`。

**超时与硬限制**。本地 `requestTimeout` / `sseReadTimeout` 均配 300 秒，但平台侧"任务规划套件-结果渲染"环节存在约 180 秒的硬超时，超过即断流并返回不完整响应。为此本地实现两道抢救：`_repair_truncated()` 对被截断的 JSON 补齐闭合结构后再解析，`_fix_escapes()` 修复 LaTeX 命令被当作非法转义导致的解析失败。

### 2.4 通信通道之二：内存消息总线（横向，智能体 ↔ 智能体）

智能体之间的传话由 `core/message_bus.py` 承担，提供 `register` / `post` / `fetch` 三个方法；AgentNode 对外暴露 `send()` / `fetch_messages()` 两个动作。消息结构为 `{ "from", "to", "type", "content", "ts" }`，按收件人姓名路由到内存邮箱，取完即清。

在用的消息类型：

| type | 方向 | 场景 |
|---|---|---|
| `derivation` / `feedback` | math ↔ verify | 快问快答的双节点校验 |
| `sub_question` / `answer` | 编排器 ↔ 专家 | 流水线第 3 步并行咨询 |
| `summary` | lit → 编排器 | 文献综述回传 |
| `verdict` / `correction` | verify ↔ 编排器/专家 | 评级与闭环修正 |
| `angle@<专家>` | 规划者 → 编排器 | 拆解出的每个子问题 |
| `digest@<专家>` / `global` | report → 编排器 | 分节统稿 / 全局统稿结果 |

总线是逻辑信箱而非网络组件：纯 Python 进程内 `defaultdict(list)`，不落盘、不走网络。它解决的是消息语义与投递记录，真正的数据搬运发生在编排器的函数调用里。这一取舍使整条流水线可以在断网环境下用 mock 完成单元测试。

同时 `_record()` 会把每次投递的 `(step, from, to, type, chars)` 追加进 `pipeline.steps`，最终写进报告附录，使通信过程本身成为交付物的一部分。

### 2.5 通信通道之三：工具回环协议（智能体 ↔ 本地工具）

平台智能体无法自行充当 MCP 客户端，因此本项目自定义了一套纯文本协议，使智能体能够驱动本地工具。

**请求格式**（写在智能体回答里，一行 JSON，用中文方括号包裹）：

```
〔工具请求〕{"name": "sympy_check_equality", "arguments": {"left": "...", "right": "..."}}〔/工具请求〕
```

不使用标准代码围栏，原因是平台 SSE 对围栏的截断处理会打断流，中文标记不会被围栏解析器吞掉。

**结果回注格式**：

```
〔工具结果 run_python | 执行成功〕{"returncode": 0, "stdout": "..."}〔/工具结果〕
```

**回环控制参数**（`core/tool_driver.py`）：

| 参数 | 取值 | 作用 |
|---|---|---|
| `MAX_TOOL_ROUNDS` | 2 | 一个问题最多两轮工具回环 |
| `MAX_CALLS_PER_ROUND` | 3 | 单轮最多执行 3 个请求 |

| 参数 | 取值 | 作用 |
|---|---|---|
| `RESULT_MAX_CHARS` | 2500 | 单个工具结果注入提示词的上限 |
| `MAX_PREFLIGHT` | 2 | 每次提问最多预跑 2 个离线工具 |
| 上下文重建 | 原问题 + 预跑结果 + 工具说明书 + 上轮回答（截 4000 字）+ 工具结果 | 补偿会话无记忆 |
| 定稿条件 | 回答中不再出现工具标记 | 剥除标记后返回 |

**确定性预跑。**工具回环若完全依赖模型自愿申请，实际执行率极低。因此驱动器在发出首轮提问之前，先按"专家权限 + 题干关键词"本地真跑 1~2 个参数可自动推导的离线工具（`comm_lib` 标准算法库、`protocol_lookup` 本地知识库），把真实数值以"系统已真实执行"段落压进提示词并要求模型引用而非覆盖。预跑以 `round=0` 事件留痕，与自愿申请的 `round≥1` 区分；预跑失败的结果不进提示词，只写日志。`run_python` / `run_verilog` 需要模型提供代码，仍走自愿申请通道。

**最小权限白名单**（`tools/registry.py`，未列出的智能体退化为普通问答）：

| 智能体 | 可用本地工具 |
|---|---|
| math_agent | sympy_compute、sympy_check_equality、comm_lib、protocol_lookup、search_papers |
| sim_agent | run_python、comm_lib、sympy_compute、protocol_lookup、search_papers |
| hw_agent | run_verilog、comm_lib、sympy_compute、protocol_lookup、search_papers |
| comm_agent | protocol_lookup、comm_lib、sympy_compute、search_papers |
| verify_agent | 六个工具全开（审稿需要自行复算） |
| lit_review_agent | 无（自身即检索器，内部直调工具） |
| report_agent | 无（只统稿，不产生新事实） |

**容错解析。**模型写坏 JSON 属高频事件，提取器先用 `raw_decode` 严格解析，失败则退化为正则抠出工具名、参数置空，由注册层回一句中文的参数说明引导下一轮改对。工具执行的任何异常都被翻译成中文结果返回，工具永远有回复，流水线不会因一个坏参数卡死。

## 2.6 可靠性设计：故障 → 检测 → 处置

| 故障 | 检测方式 | 处置策略 |
|---|---|---|
| SSE 终答为空或只剩收尾语 | 长度阈值 + 思考链是否非空 | 回退取 `thought.delta`；调用侧注入完整作答指令 |
| 平台 180 秒硬超时截断 | JSON 解析失败 / 结尾无闭合 | 截断抢救 + 分段统稿规避 + 本地兜底 |
| 统稿返回非法 JSON | `_parse_draft` 关键字段判空 | 转义修复 → 重试 → 原文直录 → 机械拼装，四层不退空 |
| 专家回答失败 | 并发 future 捕获异常 | 只警告不中断，其余专家继续，报告标注缺席 |
| 校验环节不可用 | verify 两次异常 | 降级为"校验被跳过"，报告必出 |
| 工具坏参数 | TypeError / ValueError | 转中文提示回喂，引导下一轮改对 |
| 子进程绘图阻塞 | 预防式 | 强制 `MPLBACKEND=Agg` |
| 网关限流 | 连接重置异常码 | 并发限 2 + 退避重试 |
| 仿真代码提取不到 | 四形态提取均失败 | 记录"无法真执行"，不编造运行结果 |

### 2.7 配置与可观测

- **配置**：`config.json` 集中管理平台地址、APP_KEY、两个超时、7 个智能体的 agentCode / agentVersion / kbIds 与个性化交付指令；`ConfigManager` 支持 Web 设置页读写与校验。
- **日志**：`structlog` 结构化事件（`agent.init`、`bus.post`、`tool.executed`、`pipeline.*.failed` 等）落 `logs/session.jsonl`。
- **轨迹**：`pipeline.steps` 全量记录消息，报告附录直接呈现。

# 03 三个核心机制

本章说明三项关键机制：仿真真执行、校验真闭环、统稿保内容。

## 3.1 机制一：仿真真执行闭环

**原则**：凡可本地计算的，一律本地真算，算完把产物嵌进报告。仿真不是"给一段代码看看"，误码率曲线必须由真实执行产生。

| 线路 | 谁产出代码 | 谁执行 | 执行器 | 产物去向 |
|---|---|---|---|---|
| 数值仿真 | sim_agent 回答中的 Python 代码块 | 流水线第 3.5 步 `_execute_sim_code` | 本地子进程（`tools/sim_tools.execute_python_code`，超时 120 秒，强制 `MPLBACKEND=Agg`） | PNG 转 base64 嵌入报告仿真图表区 |
| 硬件验证 | hw_agent 回答中的 Verilog | 本地工具 `run_verilog` | Verilator / iverilog 本机编译执行 | 编译与仿真结论回注下一轮 |

实现要点：代码提取与执行逻辑收敛在工具层，与 `run_python` 共用同一套安全执行实现；执行失败不伪装成功，事件 `pipeline.sim.exec_failed` 记标准错误输出，报告里该节就没有图；未提取到代码记事件 `pipeline.sim.no_code` 留痕；硬件仿真选择本地 Verilator 而非外接仿真服务，以满足内网部署下执行链最短、可复现、不引入新依赖的要求。

## 3.2 机制二：三级校验与闭环修正

多专家并行作答必然出现口径不一致、量纲错误、规范版本混用，只校验不修正等于把问题原样交给读者。

1. **三级评级**：verify_agent 的校验提示词硬规定只能给【通过】/【有条件通过】/【不通过】三档，并逐条写理由（量纲检查、单调性、特殊值、文献对照等）；
2. **闭环修正**：评级为【不通过】时，流水线把校验意见拆成反馈发回对应专家重答并重新校验，最多 `MAX_CORRECTION_ROUNDS = 2` 轮（`_verify_with_correction`），事件 `pipeline.correction.reverify` 留痕；
3. **校验者自己也有工具**：verify_agent 白名单含恒等式复核、Python 执行、通信算法库等六个工具，可自行复算；
4. **评级进报告**：全局统稿提示词硬规定 `conclusion` 字段必须以三级评级开头并忠实转述理由，封面徽章颜色随评级变化。

## 3.3 机制三：分段统稿与高审美渲染

**问题**：平台单次调用有约 180 秒硬超时，而七智能体的原始材料动辄上万字。把全部材料塞进一次统稿调用并压缩配额，会使专家论证在报告正文中所剩无几。

**方案：分段统稿**，把一次大调用拆成"每位专家 1 次分节统稿 + 1 次全局统稿"：

| 子任务 | 输入 | 输出 | 配额 |
|---|---|---|---|
| 分节统稿 ×N | 单专家完整论证（上限 6000 字）+ 其子问题 | `findings` 仅 1 条 | 正文 600~800 字，保留全部关键公式、数值与规范编号 |
| 全局统稿 ×1 | 文献综述截 1800 字 + 每专家摘要 260 字 + 校验截 900 字 | 摘要/背景/交叉/结论/展望/关键词 | 按角色手册满额 300/400/400/300/200 字 |

单次调用的输入输出都变小，从而避开硬超时；分节统稿用 2 线程并发（`ThreadPoolExecutor`）控制总时长，且每次调用开独立会话以规避平台同会话并发锁。

**三层兜底**：单节统稿失败 → `_digest_fallback` 原文直录（上限 2500 字，标记 `degraded`）；全局统稿失败 → 本地补齐全局字段，分节内容不受影响；平台全挂 → `_fallback_draft` 本地机械拼装七字段。另有三道解析保险：`_repair_truncated`（截断 JSON 补闭合）、`_fix_escapes`（非法转义双写）、`restore_latex_ctrl`（还原被 JSON 吃掉的反斜杠，公式不露源码）。

**渲染层**：报告为单文件 HTML，靛蓝瓷主题（深蓝封面 + 金色章号 + 衬线标题）、KaTeX 公式、代码高亮、仿真图内嵌、每节末尾"专家原文全貌"可展开折叠区、附录协作过程留痕表。排版库依赖外网时存在退化风险，故内置本地 Markdown 渲染器（覆盖标题、表格、引用、列表、代码围栏、粗斜体），外网库不可达时自动接管。

### 3.4 六道质量门

三个机制在一次科研任务中的咬合顺序：拆解 → 文献 → 专家作答（工具预跑 + 工具回环）→ 仿真真执行 → 三级校验与闭环修正 → 分段统稿与渲染。在此之上设置六道质量门：

| 质量门 | 实现 |
| --- | --- |
| ① 拆解覆盖门 | 拆解结果必须 ≥3 题且 ≥3 类专家；不合格带补正指令重问一次；仍不合格按缺失视角本地回填模板子问题 |
| ② 工具优先门 | 工具说明书压在"一次性输出"后缀之后，并附工具优先指令：先申请工具、后给终答 |
| ③ 独立会话门 | 每次 `think()` 开独立会话，会话 ID 局部化，支持多线程并发调用同一智能体 |
| ④ 本地排版门 | 内置本地 Markdown 渲染器兜底，外网库不可达时正文不退化为裸文本 |
| ⑤ 工具预跑门 | 驱动器按权限与关键词先本地真跑离线工具，把真实数值压进首轮提示词；两个离线工具权限扩至全体专家 |
| ⑥ 公式卫生门 | 解析后按命令白名单还原被吃反斜杠；`\text{}` 内下划线转为 KaTeX 可接受写法；渲染前 `sanitize_math` 统一收口 |

门②依赖提示词、门⑤不依赖模型自觉，二者构成同一通道上的两层。六道门均有离线回归覆盖，见 6.4。

# 04 七个智能体逐个详述

> 每个智能体按同一张卡片写：能干什么 → 手册硬规则 → 工具用法 → 本地便捷方法 → 能力边界。平台版本号取自 `config.json` 实录。

## 4.1 总览表

| 智能体 | 平台版本 | 挂载知识库 | 本地工具白名单 | 手册 | 流水线角色 |
|---|---|---|---|---|---|
| math_agent | 1786194666467 | 数学专题库 1 个 | 符号推演、恒等式复核、公网检索 | 172 行 | 专家 / 候选规划者 |
| comm_agent | 1787757082841 | 协议库 2 个 | 协议查询、公网检索、符号推演 | 52 行 | 专家 / **默认规划者** |
| hw_agent | 1786113673445 | 硬件库 2 个 | Verilog 编译仿真、公网检索 | 42 行 | 专家 |
| sim_agent | 1787760897423 | 不挂（仿真靠本地执行） | Python 执行、通信算法库、符号推演、公网检索 | 44 行 | 专家 |
| lit_review_agent | 1787753150262 | 协议库 2 个 | 不挂（内部直调检索工具） | 55 行 | 第 2 步文献调研 |
| verify_agent | 1788075577472 | 不挂 | 恒等式复核、符号推演、Python 执行、通信算法库、公网检索 | 191 行 | 第 4 步三级校验 |
| report_agent | 1788229915815 | 不挂 | 不挂 | 59 行 + 备用汇报手册 | 第 5 步统稿 |

## 4.2 math_agent：信息论与数学推导

- **能干什么**：信道容量、误码率、频谱效率等公式推导与量级估算；恒等式与不等式证明；分贝与线性、带宽与速率的数值换算；推导结果自查与修正。
- **手册硬规则**：推导规范与"推导三步法自查"、数值计算注意事项、发送 verify 前的自检清单、约束节。
- **工具用法**：`sympy_compute` 做符号化简与求导，`sympy_check_equality` 复核恒等式，使每步可复算；`comm_lib` 提供标准误码率与容量数值作为量级参照，`protocol_lookup` 提供本地知识库公式摘录，两者由驱动器预跑。
- **本地便捷方法**：`solve_awgn_capacity(带宽, 信噪比)`、`derive_capacity_change(场景)`。
- **边界**：超长推导可能触及平台 180 秒硬超时，由截断抢救与重试处置。

## 4.3 comm_agent：3GPP 协议与组网（兼默认规划者）

- **能干什么**：规范条款查询（编号、版本、章节）、信令流程分析、组网与参数配置解读、协议演进对比；兼任流水线第 1 步的问题拆解（`PLANNER_CANDIDATES` 首位）。
- **手册硬规则**：回答必须给规范编号与版本；不确定时明说"需查规范"，不编造。
- **工具用法**：`protocol_lookup` 查本地协议知识库条目（驱动器预跑一次，命中摘录直接摆在它面前）；`comm_lib` 取标准算法库数值；`sympy_compute` 核算功率与链路预算公式。
- **本地便捷方法**：`query_protocol(问题)`。
- **边界**：规划输出受拆解覆盖门约束（≥3 题且 ≥3 类专家）；离线时规划回退 math_agent，再不行由本地回填模板子问题。

### 4.4 hw_agent：FPGA / Verilog 硬件实践

- **能干什么**：按规格生成可综合 Verilog 模块（译码器、滤波器、接口等）；时序与关键路径分析；查找表、寄存器、块存等资源开销估算；测试激励编写。
- **手册硬规则**：输出格式节（平台渲染要求）、代码规范、禁止节，以及验证闭环必做——生成后必须经 `run_verilog` 本地编译或仿真，不允许只给代码不给结论。
- **工具用法**：`run_verilog` 调本机 Verilator / iverilog，编译与仿真结论回注下一轮；`comm_lib`、`sympy_compute`、`protocol_lookup` 提供可预跑的数值与规范依据。
- **本地便捷方法**：`generate_verilog(规格)`、`explain_timing(模块代码)`。
- **边界**：平台上存在多版本并存，以 `config.json` 中版本为准；RTL 仿真依赖本机已装工具链，换机部署需先安装。

### 4.5 sim_agent：蒙特卡洛仿真与数值验证

- **能干什么**：误码率、容量、星座图、收敛性等仿真方案设计与代码生成；理论曲线对照与偏差分析；仿真五步法执行（方案 → 代码 → 运行 → 对照 → 结论）。
- **手册硬规则**：仿真五步法必做；禁止编造运行结果；输出格式固定四章节。
- **工具用法**：`run_python` 本地真跑、`comm_lib` 调通信算法库、`sympy_compute` 算理论值、`protocol_lookup` 查本地知识库；`comm_lib` 会被预跑一次，仿真开始前即取得理论基准曲线。
- **本地便捷方法**：`simulate_ber(调制方式,…)`、`simulate_capacity(信噪比范围)`、`think_verified(任务)`（生成 → 执行 → 偏差校验 → 不达标自动反馈重跑）、`execute_code(代码)`。
- **边界**：复合提示词易导致规划超时，流水线对其改用精简提示词（只给课题与子问题，不注入全文综述），并由个性化交付指令要求四章节一次性输出。

### 4.6 lit_review_agent：公网 + 知识库双源文献调研

- **能干什么**：arXiv / Semantic Scholar 公网检索与平台知识库检索双源融合；按子问题角度组织综述；生成带作者年份或规范编号标注的参考文献清单。
- **手册硬规则**：综述流程三步规范；结论必须有出处标注；无出处的论断标"待验证"。
- **工具用法**：不挂白名单，内部直调检索工具（`core/lit_search`），避免回环套回环。
- **本地便捷方法**：`survey(课题)`、`survey_with_angles(课题, 角度列表)`、`_attach_web_refs`（公网参考文献注入）。
- **边界**：公网不可达时自动退化为仅知识库源，综述标注来源受限；检索源限流时进入冷却并切换可用源。

### 4.7 verify_agent：三级校验

- **能干什么**：对 math / comm / hw 产出做原理性审查——量纲检查、单调性、特殊值、文献对照、规范版本核对、代码可综合性判断；给出三级评级与逐条理由。
- **手册硬规则**：通用四项清单 + 三类专项检查；回复格式硬规定三档评级模板；禁止"看起来没问题"式空话。
- **工具用法**：六个工具全开，可自行复算恒等式、重跑仿真片段、调标准算法库取基准值；校验前驱动器按课题预跑一次，使其先握有真实数值再读专家结论。
- **本地便捷方法**：`check_derivation(内容, 主题)`。
- **边界**：校验输入为综述截 1000 字 + 每专家截 1500 字（约 4200 字），超长课题会损失细节。

### 4.8 report_agent：统稿主编

- **能干什么**：把多智能体产出统稿为固定七字段 JSON（摘要、背景、核心发现、交叉分析、结论、展望、关键词）；支持分节统稿与全局统稿两种子任务；遵循编辑方针六条（结论先行、忠实原文、保留评级、交叉印证、缺口诚实、公式规范）。
- **手册硬规则**：输出必须是单个合法 JSON、不含 JSON 以外文字；`conclusion` 必须以三级评级开头；禁止编造材料中没有的数据。
- **工具用法**：不挂工具，统稿不产生新事实。
- **本地便捷方法**：`compile_report(标题, 材料)`。
- **备用手册**：`report_presentation_system.md`（组会与答辩文本模式，四步汇报流程），与在线的 JSON 统稿协议不混用。
- **边界**：平台侧存在 180 秒硬超时与同会话并发锁两类间歇限制，分别由分段统稿与独立会话门处置，另有三层兜底保证报告必出。

### 4.9 分工边界：谁向谁要什么

| 求助方 | 被求方 | 要什么 | 通道 |
|---|---|---|---|
| 编排器 | comm / math | 课题拆解 | 平台调用 |
| 编排器 | 四专家 | 子问题论证 | 平台调用 + 工具回环 |
| 编排器 | verify | 三级校验 | 平台调用 + 工具回环 |
| 编排器 | report | 分节与全局统稿 | 平台调用 |
| math / comm / hw | verify | 反馈修正 | 编排器代投递（总线） |
| 任意专家 | 本地工具 | 真实计算、检索、仿真 | 驱动器预跑（round=0）+ 工具回环协议（round≥1） |

智能体之间没有直接对话通道，一切横向消息由编排器代投递并留痕，这是星型拓扑的硬约束，也是协作过程可追溯的前提。

---

# 05 平台资产盘点

> 本章为资产总账：平台侧智能体、知识库、技能，本地工具、提示词、知识资产与评测资产。数据以 `config.json`、`tools/registry.py` 与目录实录为准。

## 5.1 智能体资产（7 个，全部在线）

见 4.1 总览表。三点补充：七个智能体均为平台角色对话型智能体，由本地 `AgentNode` 统一封装会话创建、SSE 消费、重试、知识库挂载与思考链兜底，本地不重复实现任何平台能力；版本号全部写入 `config.json`，换版本只改配置不改代码；hw_agent 在平台上多版本并存，以配置中的版本为唯一准绳。

## 5.2 知识库与本地知识资产

平台侧绑定 5 个知识库：

| 知识库 ID | 挂载者 | 内容 |
|---|---|---|
| t6doo3pjacqe | math_agent | 数学专题库（信息论、容量、误码率） |
| jka3mxb19y08 | comm_agent、lit_review_agent | 协议规范库其一 |
| anb7bpimm2jn | comm_agent、lit_review_agent | 协议规范库其二 |
| jfoi1hlf60e7 | hw_agent | 硬件实践库其一 |
| d8xgrvwu8zr7 | hw_agent | 硬件实践库其二 |

sim、verify、report 不挂库：仿真靠本地真跑、校验靠工具复算、统稿靠输入材料，属设计选择。

本地知识资产（`knowledge/`，即导入平台的源文件）：`knowledge/math/` 含 9 个专题与 4 件权威文献（Shannon 1948 原文 PDF、Shannon 全文文本、两份讲义）；`knowledge/sim/` 含 7 篇仿真方法学文档。`build_kb_json.py` 可把它们打成平台导入包（`outputs/sim_kb_import.json`），知识库可一键重建。

## 5.3 本地工具层（7 个工具，最小权限白名单）

| 工具 | 中文名 | 作用 | 可用者 |
|---|---|---|---|
| sympy_compute | 符号数学推演 | 化简、求导、积分、解方程 | math、sim、comm、hw、verify |
| sympy_check_equality | 恒等式复核 | 判断两式是否恒等 | math、verify |
| search_papers | 公网文献检索 | arXiv / Semantic Scholar | 除 report 外全部 |
| run_python | Python 仿真执行 | 沙箱子进程运行代码取 stdout 与图 | sim、verify |
| comm_lib | 通信算法库 | 调制、编码、信道标准实现 | 除 lit_review、report 外全部（可预跑） |
| run_verilog | Verilog 编译仿真 | Verilator / iverilog 本机执行 | hw |
| protocol_lookup | 协议知识查询 | 本地协议条目检索 | math、sim、comm、hw、verify（可预跑） |

设计原则：统一"名称、描述、参数、执行函数"四要素注册（`@tool` 自注册），未来可平移到平台工具中心而注册层不动；任何异常都转为中文错误结果，工具永远有回复。标"可预跑"的两个工具参数可自动推导、离线、无副作用，因此能不依赖模型自觉就把真实数值送进回答。

### 5.4 技能资产（3 个，已上传平台技能中心）

`sim-five-step-method`（仿真五步法）、`research-report-compile`（统稿编辑方针）、`research-report-aesthetic`（靛蓝瓷报告审美规范）。

上传链路已实测跑通（技能中心 zip 上传）。实测结论：角色对话型智能体不支持挂载空间技能，因此三个技能在运行时以手册资产形态生效（内容并入各智能体手册与本地渲染器），不构成运行时依赖。

### 5.5 提示词资产（13 份手册）

系统手册 7 份（math / comm / hw / sim / lit_review / verify / report，含编辑方针六条与七字段协议）；调用约束 5 份（constraints_math / comm / hw / verify / report，规定输出格式、禁止事项、知识库引用要求）；备用 1 份（report_presentation_system，组会与答辩文本模式）。平台在线手册与本地 `prompts/` 双份维护，交付以本地为准。

### 5.6 评测与留痕资产

评测集 `evals/tasks.json` 为 32 道单题 + 6 道协作题共 38 题，三类判分（直接比对、裁判智能体、协作链路）；跑批器为 `tests/run_eval.py`、`tests/merge_eval_results.py`、`tests/make_score_sheet.py`；留痕为 `logs/session.jsonl` 全量结构化事件、`docs/验证记录图片/` 平台与 Web 截图、`outputs/research_*.html` 历次真实产出。

---

# 06 测试与量化验证

> 本章只列做过的测试轮次与结果。

## 6.1 测试轮次总账

| 轮次 | 测试内容 | 执行方式 | 结果 |
|---|---|---|---|
| 1 | 能力评测集摸底（38 题集，实跑 37 题） | `tests/run_eval.py` 在线跑批 + `merge_eval_results.py` 合并 | **35/37 通过（94.6%）** |
| 2 | 离线回归矩阵（6 套断言） | `tests/test_*.py` 本地直接运行 | **33 / 38 / 21 / 15 / 11 / 51 全部通过** |
| 3 | BPSK 端到端流水线 | `tests/test_pipeline.py` 真跑六步 | **11/11 断言通过**，报告内嵌真跑仿真图 |
| 4 | 平台资产与 Web 六页逐页核对 | 平台控制台实查 + 浏览器逐页 | **29 张核对截图**，7 智能体在线、5 知识库绑定、3 技能上传 |
| 5 | 综合多智能体跑批（三道跨领域课题） | `tests/run_multi_agent_check.py` | 三道跨领域课题，21 项硬校验逐条判定（设计见 6.5） |
| 6 | 人工评分复核 | `tests/make_score_sheet.py` 生成四维打分表 | 评分表 1 份，逐题复核判分合理性 |

## 6.2 评测集设计（38 题）

单题 32 道（`evals/tasks.json`）覆盖七智能体各自职责域，判分两档：直接比对（关键词、数值、规范编号命中）与裁判智能体评分（verify_agent 当裁判）；协作题 6 道，考"甲答完交给乙再审"的链路。分布为 comm 6、hw 6、math 6、lit 5、sim 2、report 2、裁判类 5、协作 6。

## 6.3 摸底成绩：35/37

跑批时间 2026-09-02 13:17，实跑 37 题（collab-03 未跑）。

| 类别 | 题数 | 通过 |
|---|---|---|
| 单答 · comm | 6 | 6 |
| 单答 · hw | 6 | 6 |
| 单答 · math | 6 | 6 |
| 单答 · lit | 5 | 5 |
| 单答 · sim | 2 | 2 |
| 单答 · report | 2 | 1 |
| 裁判 · verify | 5 | 5 |
| 协作链路 | 5 | 4 |
| **合计** | **37** | **35（94.6%）** |

## 6.4 离线回归矩阵

不依赖平台，秒级跑完，用于保护已修复的行为不回退。

| 脚本 | 断言 | 保护对象 |
|---|---|---|
| `tests/test_tools.py` | 33/33 | 工具四要素注册、执行、异常转中文结果、工具调用协议解析 |

| 脚本 | 断言 | 保护对象 |
| --- | --- | --- |
| `tests/test_tool_preflight.py` | 38/38 | 确定性预跑：关键词识别、工具选型、真实执行与提示词注入、与自愿申请共存 |
| `tests/test_compile_split.py` | 21/21 | 分段统稿配额、三层兜底、合并顺序、降级标记 |
| `tests/test_plan_coverage.py` | 15/15 | 拆解质量门：重问、回填、顶替、轨迹留痕 |
| `tests/test_latex_escape.py` | 51/51 | 公式卫生：反斜杠还原、`\text{}` 内下划线、真换行与真制表符不误伤、渲染 HTML 无残渣 |
| `tests/test_pipeline.py` | 11/11 | BPSK 端到端：拆解、综述、专家、校验、统稿、报告内嵌图 |
| `tests/test_sim_templates.py`、`tests/test_report_agent.py` | 存在 | 仿真模板与统稿协议解析的专项回归 |

## 6.5 综合多智能体跑批

单题评测考不出协作能力，因此另设一套跨领域课题跑批：三道课题各自天然横跨原理推导、规范依据、硬件落地、数值验证四类视角，配合拆解覆盖门逼出多位专家真实参与。

| 课题 | 内容 | 涉及的四类视角 |
| --- | --- | --- |
| 一 | 5G NR 上行共享信道（PUSCH）链路预算与覆盖分析 | 香农容量理论界 / 3GPP 功率与调制参数 / 蒙特卡洛覆盖仿真 / 射频前端硬件约束 |
| 二 | Massive MIMO 信道估计 | MMSE 与 LS 误差理论 / 3GPP 参考信号规定 / FPGA 复杂度与资源开销 / 估计精度仿真对照 |
| 三 | eMBB 场景 Polar 码与 LDPC 码选型 | 纠错性能理论界 / 3GPP 标准化现状 / 译码器硬件开销与时序 / 误码率仿真验证 |

21 项硬校验由脚本内 `check()` 逐条判定，不依赖人工读数。每道课题 6 项：拆解覆盖 ≥3 类专家、实际作答 ≥3 类专家、每位专家论证 ≥200 字、校验结论含三级评级、统稿核心发现 ≥3 节且 ≥1500 字、报告存在且带本地排版兜底；三题合计再判 3 项：四类专业专家全部真实参与、工具回环至少触发一次、每道课题都产出报告。退出码 0 表示全通过。单题异常只记 FAIL 并继续下一题，输出实时落盘 `_multi_out.txt`。

## 6.6 平台侧核对留痕（29 张截图，`docs/验证记录图片/`）

| 类别 | 张数 | 内容 |
| --- | --- | --- |
| 平台控制台 | 9 | 登录页、hw_agent 版本核对 3 张、技能详情与上传链路 4 张、共享空间 |
| Web 六页 | 13 | 首页问答、流水页、报告库、监控、设置逐页核对（侧边栏态 7 张 + 桌面态 6 张） |
| 网络排查 | 2 | 连接错误区分、公网检索真实响应 |
| 文档核对 | 3 | 汇报书封面、正文、页眉页脚逐页检查 |
| 仿真产物 | 1 | BPSK 误码率对照图（本地真跑产物） |
| 运行探测 | 1 | 流水线探测截图 |

# 07 后续规划

本项目的边界在结项时是清楚的：智能体的能力覆盖到理论界、协议依据、硬件实现与误码率级仿真，部署形态受内网约束。以下两条是已论证过可行性、列为下一阶段的工作，均不改变现有六步流水线与六道质量门的结构。

## 7.1 把物理层信号处理链路纳入研究与验证范围

当前四位专家在物理层内部仍以结论引用为主，信号处理环节缺少真跑验证。下一阶段补齐四个环节：

| 环节 | 纳入方式 | 落到哪个智能体 |
| --- | --- | --- |
| 波形生成与解调（CP-OFDM / DFT-s-OFDM） | 仿真模板新增波形级用例，输出星座图与 EVM | sim_agent |
| 信道估计与均衡（LS / MMSE） | 与 Massive MIMO 课题打通，估计误差进链路预算 | sim_agent、math_agent |
| 信道编译码（Polar / LDPC 译码） | 误码率曲线从调制级扩展到编码级，含译码迭代次数 | sim_agent |
| 对应 RTL 模块（FFT、均衡器、译码器核） | 沿用“进过编译与仿真”的交付标准，补资源与时序报告 | hw_agent |

配套动作：math_agent 的公式库补物理层条目（EVM 与星座误差关系、MMSE 均衡闭式解、LDPC 密度演化界），入库前照例跑真实检索验证；校验智能体的检查项同步增加“仿真环节与协议波形是否对齐”一条，避免再现本报告校验意见中指出的 PAPR 分析未区分波形的缺口。

## 7.2 公网部署形态

系统目前只能部署在内网，根因是智能体平台接口仅内网可达，这一约束在结项时不解除。公网化分两步，均不改动编排逻辑：

1. **静态资产先行**：研究报告、知识库源文档、评测结果改为公网只读发布（容器化静态服务加访问令牌），评审与协作方无需进内网即可查阅与复核。
2. **调用链路反向化**：公网侧只部署 Web 应用与编排器，对平台的全部调用改由内网侧主动建立的出站长连接承载，公网请求经该通道转发，平台接口不暴露到公网；同时补齐三件事——应用密钥按环境分离与定期轮换、按来源白名单与速率限制放行、调用留痕与本地一致地落盘可审计。

两步的共同前提是先完成容器化（仓库已含部署定义），并在公网环境重跑一遍离线回归与端到端验收，确认渲染兜底与超时重试在弱网下仍然成立。

# 附录 A · 最终提交材料里有什么

> 交付包根目录 `最终提交材料/`，共 152 个文件、约 7.7 MB，四个一级条目。过程性材料（运行日志、评测原始结果、验证截图、排查记录、历史存档）保留在仓库 `中间材料/` 与 `logs/`，不随包提交，需要时按附录 B 的路径查阅。> 整个包由 `build_submission.ps1` 一键从仓库同步，包内每个文件都能在仓库里找到源；多余文件由 `clean_submission.ps1` 按白名单清理（默认只预览，加 `-Yes` 才真删）。

## A.1 一级结构

| 条目 | 体量 | 是什么 |
|---|---|---|
| `提交材料清单.md` | 1 份 | 评审入口：阅读顺序建议 + 目录说明 + 关键事实速览 |
| `项目文档\` | 7 份 · 2.4 MB | 结课汇报书三态（MD 源 / HTML 屏幕态 / PDF 打印态）+ 架构图 + 两份历史文档 + 构建脚本 |
| `演示成果\` | 7 份 · 0.8 MB | 系统真实产出：4 份研究报告 HTML + 3 张本地真跑的仿真图 |
| `代码\` | 137 份 · 4.6 MB | 完整可运行系统：编排层、智能体角色、工具、Web、提示词、知识库源文档、技能源、评测与测试 |

### A.2 项目文档\

| 文件 | 说明 |
|---|---|
| `项目结课汇报书.md` | **唯一内容源**：本书正文（6 章 + 2 附录） |
| `项目结课汇报书.html` | 屏幕阅读态，双击即开（封面 + 目录 + 正文，靛蓝瓷主题） |
| `项目结课汇报书.pdf` | 打印态 A4，页眉页脚带页码，每章另起一页 |
| `build_report_pdf.py` | 由 `.md` 一键重建 HTML 与 PDF；加 `--check` 会逐页统计字符数自检空页 |
| `architecture.svg` | 系统分层架构图（矢量） |
| `AI_for_Comm_项目方案书.pdf` | 立项阶段方案书（历史沿革，口径以本书为准） |
| `AI_for_Comm_实践手册.pdf` | 平台操作手册（历史沿革） |

三态同源：HTML 与 PDF 都由 `.md` 渲染而来，改完文稿跑一次构建脚本两者同时更新。

### A.3 演示成果\

| 文件 | 体量 | 内容 |
|---|---|---|
| `research_20260901_234713.html` | 228 KB | BPSK 在 AWGN 信道下的误码率性能与频谱效率验证 |
| `research_20260902_003653.html` | 173 KB | 同课题复跑 |
| `research_20260902_215747.html` | 120 KB | LDPC 码在 5G NR 中的应用（第二个题材） |
| `research_20260902_225648.html` | 42 KB | 5G NR PUSCH 链路预算与覆盖分析（跨领域题材） |
| `bpsk_ber_awgn.png` | 78 KB | 蒙特卡洛误码率曲线（本地沙箱真跑产物） |
| `ber_comparison.png` | 64 KB | 多方案误码率对照 |
| `bpsk_awgn_ber.png` | 63 KB | 仿真值与理论值对照标杆图 |

四份报告均为单文件 HTML、离线双击可开：KaTeX 公式、代码高亮、仿真图内嵌、每节末尾可展开专家原文、附录带协作过程留痕。

### A.4 代码\

| 目录 | 文件数 | 内容 |
| --- | --- | --- |
| （根） | 8 | `main.py` CLI 入口、`config.json` 平台与智能体配置、`requirements.txt` 、`Dockerfile` / `docker-compose.yml` / `.dockerignore` 、`build_kb_json.py` 知识库导入体构建、`目录结构说明.md` （末节含运行方式） |
| `core\` | 12 | 本地编排层：流水线、编排器、智能体节点、消息总线、工具驱动器、报告渲染、文献检索、配置管理、日志 |
| `agents\` | 9 | 七位专家智能体的本地适配子类 |
| `tools\` | 9 | 7 个本地工具 + 注册表与最小权限白名单 |
| `prompts\` | 14 | 7 份角色手册 + 6 份硬约束 + 1 份备用手册 |
| `web\` | 23 | Flask 应用、6 条路由、6 个页面模板、6 个前端脚本、样式与 logo |
| `knowledge\` | 21 | 数学与仿真两个本地知识库的源文档 |
| `skills\` | 4 | 3 个平台技能的源 `SKILL.md` |
| `evals\` | 8 | 38 题评测集与说明 + `results\` 6 份评测原始结果 |
| `tests\` | 29 | 离线回归矩阵、评测跑批器、综合多智能体跑批脚本 |

包内不含 `logs\` 、`outputs\` 等运行时产物目录；首次运行按 `目录结构说明.md` 末节启动即可自动重建。

# 附录 B · 数字复核索引

> 本书每个数字都对应一个可执行或可打开的凭据，路径均相对仓库根目录。

| 数字 | 凭据位置 | 复核方式 |
| --- | --- | --- |
| 7 个智能体与平台版本号 | `config.json` 的 `agents` 段 | 逐个对照 `agentCode` / `agentVersion` / `kbIds` |
| 5 个知识库、3 个技能 | 平台接口实查 | `docs/验证记录图片/` 对应截图 |
| 38 题评测集 | `evals/tasks.json` | 数条目：32 单题 + 6 协作题 |
| 摸底 35/37 | `中间材料/eval_20260902_131712.csv`、`eval_summary_20260902_131848.csv` | `tests/merge_eval_results.py` 合并口径可复算 |
| 离线回归 33 / 38 / 21 / 15 / 11 / 51 | `tests/test_*.py` | 逐个直接运行，秒级出断言计数 |
| 工具预跑真实执行 | `logs/session.jsonl` | 过滤 `tool.preflight` 与 `round=0` 的 `tool.executed` 事件 |
| 报告体量、内嵌图与展开块 | `outputs/research_*.html` | 看文件大小与页面内嵌图、折叠区数量 |
| 公式不露源码 | `tests/test_latex_escape.py` | 51 条断言含"渲染后 HTML 无控制字符残渣" |
| 平台 180 秒硬超时、同会话并发锁 | 平台 trace 与本地日志 | `logs/session.jsonl` 的 `agent.think.failed` |
| 综合跑批 21 项硬校验 | `_multi_out.txt` | 脚本逐条打印 `[PASS]` / `[FAIL]`，末行给总计数 |
| 交付包文件数与体量 | `最终提交材料/` | 目录属性统计，或运行 `build_submission.ps1` 看校验段输出 |

复核顺序建议：先跑离线回归（不依赖平台、秒级完成），再看 `logs/session.jsonl` 的事件计数，最后打开报告 HTML 对照正文与图表；平台侧资产以截图与接口实查为准。